
Machine Learning Based Analog Circuit Fault Diagnosis
PDF 6 — Chapter 11 + Chapter 12: ML Fundamentals & SVM Theory
What ML Actually Learns | Decision Boundaries, Margins, Kernels, Implementation

CHAPTER 11 — Machine Learning Fundamentals

Chapter 11 — Machine Learning Fundamentals (Before Learning SVM) Before Starting Congratulations. Till now we have completed the entire electronics side of the project. We have learned

[OK] Analog Circuits

[OK] LTspice

[OK] Dataset Generation

[OK] Feature Extraction

[OK] All 17 Features

Question. What happens next? Most students answer Train SVM. Wrong. Before learning SVM, we must first understand what

Machine Learning

actually does. Otherwise SVM will look like magic. This chapter removes that magic.

What you will know after finishing this chapter

After finishing this chapter, you should be able to answer [OK] What is Machine Learning? [OK] Why do we need it? [OK] What exactly is the computer learning? [OK] Why is our problem a classification problem? [OK] What are features and labels? [OK] How does training actually work?

Difficulty

Basics 5/5

Theory 5/5

Coding 1/5

Viva Importance 5/5

Let's Begin

What is Machine Learning? Simple definition.

Machine Learning

is a technique that allows a computer to learn patterns from data without writing explicit rules.

PROFESSOR ASKS

What is Machine Learning?

EXCELLENT ANSWER

Machine Learning is a branch of Artificial Intelligence that enables computers to learn patterns from historical data and make predictions on unseen data without being explicitly programmed for every situation. Remember the words learn patterns Not memorize answers. What Does the Computer Learn? Very important question. Question. Does the computer learn electronics? No. Question. Does it learn Ohm's Law? No. Question. Does it know what a resistor is? No. Then what exactly does it learn? It learns relationships. Example Suppose our feature vector is

Peak = 5.2

RMS = 3.8

Mean = 2.9

. DC Level = 4.9 Label — RC_Normal. Another feature vector

Peak = 4.8

RMS = 3.1. DC Level = 0.001 **Label** — RC_Open. Question. What is the computer learning? It learns that certain combinations of features usually belong to certain fault classes. That's all.

Very Important Sentence

The computer does not learn circuits. It learns the relationship between feature vectors and fault labels. Remember this. This sentence alone can answer many viva questions.

Real Life Analogy

Imagine a doctor. Question. Can the doctor see the virus directly? Usually No. Instead the doctor sees Fever Cough Headache Weakness Then says Probably Viral Fever.

Machine Learning

does the same thing. It sees Peak RMS

Rise Time

DC Level. Then predicts RC_Open Exactly the same process. What Are Features? We already know this. Features are measurements taken from the waveform. Examples Peak RMS Mean

Rise Time

Energy. Question. How many features do we have? Exactly These become the input. What Are Labels? Labels are the correct answers. Example

Feature Vector

RC_Normal

Feature Vector

RL_Open

Feature Vector

RLC_Fault These labels are what the model tries to predict.

PROFESSOR ASKS

What are features and labels?

EXCELLENT ANSWER

Features are the numerical input variables extracted from the waveform, while labels represent the corresponding fault classes that the model learns to predict. Why Is Our Problem Classification?

Machine Learning

has many types. Question. Are we predicting temperature? No. Question. Stock price? No. Question. House price? No. Those are Regression problems. Question. Are we predicting one out of many fault classes? Yes. That is Classification.

PROFESSOR ASKS

Why is your project a classification problem?

EXCELLENT ANSWER

Our project predicts one fault class from a predefined set of possible fault conditions. Since the output is categorical rather than continuous, it is a classification problem.

Supervised Learning

Now another important term. Question. Do we know the correct label during training? Yes. Example **Features** — RC_Open. Known. Question. Does the computer learn using these correct answers? Yes. That is Supervised Learning.

PROFESSOR ASKS

What is Supervised Learning?

EXCELLENT ANSWER

Supervised learning is a machine learning approach in which the model learns from labeled input-output pairs provided during training. Training Question. What happens during training? Simple. The model looks at thousands of examples. Example **Features** — RC_Normal. Again. **Features** — RC_Open. Again. **Features** — RL_Normal. Again. Hundreds. Thousands. Slowly it begins to recognize patterns. That process is called Training. Testing Question. Suppose after training we give a completely new waveform. Question. Has the computer seen it before? No. Question. Can it still predict correctly? Hopefully Yes. That is Testing.

PROFESSOR ASKS

Why do we need testing?

EXCELLENT ANSWER

Testing evaluates the model's ability to generalize to previously unseen data rather than simply memorizing the training samples. Does Machine Learning Memorize? Very important. Question. Suppose the model memorizes every training sample. Question. Is that good? No. Because real circuits will produce new waveforms. Need Generalization. Generalization Generalization means performing well on new data. Not just training data. This is the real goal of Machine Learning.

Our Project

Now let's connect everything. Our project looks like this. Waveform

17 Features

Feature Vector

Machine Learning

Fault Class

Question. Where does

Machine Learning

begin? Exactly here. After Feature Extraction.

Common Misconception

Many students think

Machine Learning

reads the waveform. Wrong. Remember this sentence. Waveform

Feature Extraction

Feature Vector

Machine Learning

Machine Learning

never sees the waveform. It only sees 17 numbers.

One Beautiful Analogy

Imagine someone describing a person to you. They say Height 180 cm Hair Black Eyes Brown Age 22 Question. Did you see the person? No. You only saw their features. Exactly what SVM does. It never sees the actual waveform. Only its description.

Connection to Our Project

Everything we built until now was simply preparing these 17 numbers. Now

Machine Learning

takes over. Notice how much work happened before training even started. That is why I always said

Machine Learning

is mostly data engineering.

COMMON MISTAKES

MISTAKE: [X] Machine Learning learns circuits. Wrong. It learns relationships between features and labels.

MISTAKE: [X] Machine Learning memorizes examples. Wrong. It should generalize.

MISTAKE: [X] Training accuracy is enough. Wrong. Testing performance matters more.

REVISION NOTES

Remember these five ideas.

Machine Learning — Pattern Learning.

Features — Inputs.

Labels — Correct Answers.

Training — Learning.

Testing — Generalization.

MEMORY TRICK

Remember this sentence. "Features go in. Labels come out. Patterns are learned in between." That one sentence summarizes the entire purpose of Machine Learning in your project.

KNOWLEDGE CHECK

Level 1

- What is Machine Learning?
- What are features?
- What are labels?
- Why is our project a classification problem?
- What is supervised learning?

Level 2

- What exactly is the model learning?
- Why is testing necessary?
- What is generalization?
- Why doesn't the model use the waveform directly?
- What is the difference between training and testing?

Professor Level

- Explain the complete machine learning workflow in your project from feature vector to prediction.
- Why is feature engineering critical before training?
- Why is generalization more important than memorization?
- Could another classifier replace SVM without changing the feature extraction pipeline? Why?
- Why is your project a supervised multiclass classification problem rather than a regression problem?
- End of Chapter 11
- Congratulations.
- You now understand what Machine Learning is actually doing in your project.
- This chapter is important because it removes the misconception that "the computer understands circuits." It doesn't. It learns mathematical relationships between your 17 engineered features and the corresponding fault classes.

Author's Note The next chapter is one of the biggest in the entire Project Bible:

Chapter 12 — Support Vector Machine (SVM): Complete Theory from Absolute Basics to Master Level
In that chapter, we'll cover everything we discussed over many sessions: What an SVM really is Why it is called a Support Vector Machine Hyperplanes Margins Support Vectors Linear vs Nonlinear classification Kernels (Linear, Polynomial, RBF) Soft Margin vs Hard Margin Regularization parameter C Gamma Multiclass classification (One-vs-One) Why we chose SVM for your project And finally, how your implementation uses it before we move on to GridSearchCV.

CHAPTER 12 Part 1 — SVM from Absolute Basics

Chapter 12 — Support Vector Machine (SVM) Part 1 — Understanding SVM from Absolute Basics Before Starting Congratulations. Till now we have completed the entire electronics pipeline. We have completed the entire feature extraction pipeline. Now our project has reached this stage. Waveform

17 Features

Feature Vector

??? Question. What comes here? Answer Support Vector Machine. But before learning our implementation, we must first answer one question. What exactly is an SVM? Most students answer "It is a classifier." That answer is correct but far from complete. Let's understand what actually happens.

What you will know after finishing this chapter

After finishing this chapter you should be able to answer [OK] What is SVM? [OK] Why is it called Support Vector Machine? [OK] What problem does it solve? [OK] Why didn't we choose KNN or Decision Tree? [OK] Why is SVM suitable for our project?

Difficulty

Basics 5/5

Theory 5/5

Coding 1/5

Viva Importance 5/5

Let's Begin

Imagine we only have two fault classes. RC_Normal RC_Open Now suppose every waveform has only two features. Example

Peak Voltage

RMS Question. Can we draw every waveform as a point? Yes. Example

Peak ↑

|

| ● | ● | ▲ | ▲

+-----> RMS

Suppose the circles represent RC_Normal. The triangles represent RC_Open. Question. Can you separate them? Probably yes. Need one line. Classification Suppose we draw a line.

Peak ↑

|

| ● | ●

|-----

| ▲ | ▲

+-----> RMS

Question. What happened? The two classes became separated. That line is called a Decision Boundary. Remember this term. Very important.

PROFESSOR ASKS

What is a Decision Boundary?

EXCELLENT ANSWER

A decision boundary is the surface that separates different classes in the feature space. Notice I said surface. Not line. We'll understand why later. Can We Draw Many Lines? Question. Suppose our points look like this.

Peak ↑

|

| ● | ●

|

| ▲ | ▲

+-----→ RMS

Question. Can I draw this line?

|
|
|

Yes. Question. Can I draw this line? \ \ \ Yes. Question. Can I draw this one?

Again Yes. Many lines can separate the classes. Question. Which line should we choose? This is where SVM begins. The Main Idea Behind SVM SVM asks one question. Among all possible separating lines, which one is the best? Not just any line. The best line. What Does "Best" Mean? Imagine two roads.

Road A

passes very close to the edge of a cliff.

Road B

passes through the middle with lots of space on both sides. Question. Which road is safer? Obviously Road B. SVM thinks exactly the same way. Instead of choosing a line very close to the data, it chooses the line that leaves the maximum safety distance on both sides. This distance is called the Margin. Margin Question. What is Margin? Simple. Margin is the distance between the decision boundary and the nearest training samples. Example ● ●

▲ ▲ The empty space between the points and the line is the Margin.

PROFESSOR ASKS

What is Margin?

EXCELLENT ANSWER

Margin is the minimum distance between the decision boundary and the nearest training samples from either class. Excellent. Remember minimum distance. Why Maximize the Margin? Question. Suppose new data arrives. If our line is very close to one class, small noise may cause misclassification. Now imagine the line is far away from both classes. Small noise won't matter. Exactly. **Larger margin** — Better generalization. This is the heart of SVM.

Real Life Analogy

Imagine driving through a narrow street. Car

| |

Walls Very risky. Now imagine a wide road. Much safer. Margin is that extra space.

PROFESSOR ASKS

Why does SVM maximize the margin?

EXCELLENT ANSWER

Maximizing the margin improves the classifier's ability to generalize to unseen data by making the decision boundary less sensitive to small variations and noise. Excellent. Why Is It Called Support Vector Machine? Now the most famous viva question. Question. What are Support Vectors? Let's see. Imagine our data. ● ● ●

▲ ▲ ▲ Question. Which points actually determine the line? All of them? No. Only the nearest points. These nearest points touch the margin. These are called Support Vectors. Question. Suppose I remove a point far away. Will the line change much? Usually No. Question. Suppose I remove a support vector. Will the line change? Absolutely.

Support Vectors

control the classifier.

PROFESSOR ASKS

What are Support Vectors?

EXCELLENT ANSWER

Support vectors are the training samples closest to the decision boundary. They determine the optimal position of the separating hyperplane. This answer is extremely important. Why Are They Called "Vectors"? Question. Why Support Vector? Because each sample is represented as a vector. Remember our feature vector? [Peak, RMS, Mean, DC Level] Every sample is a vector. The important vectors become Support Vectors. Simple.

The Hyperplane

Until now I kept saying Line. Actually that is not correct. Question. Suppose we only have two features.

Decision Boundary

Line. Suppose three features.

Decision Boundary

Plane. Suppose 17 features like our project. Question. Can we draw it? No. It exists in 17-dimensional space. Such a boundary is called a Hyperplane.

PROFESSOR ASKS

What is a Hyperplane?

EXCELLENT ANSWER

A hyperplane is the decision boundary used by SVM to separate classes in a multidimensional feature space. Excellent.

In Our Project

Question. How many features do we have? Question. Can we draw 17 dimensions? No. But mathematically SVM creates a 17-dimensional hyperplane. That hyperplane separates our fault classes.

One Beautiful Analogy

Imagine 17 doctors examining one patient. Each doctor checks one symptom. Together they decide the disease. Similarly our SVM looks at 17 features simultaneously before predicting the fault class.

Connection to Our Project

Our pipeline now becomes Waveform

17 Features

17-Dimensional Feature Vector

SVM

Optimal Hyperplane

Fault Prediction

This is exactly what happens inside our project.

COMMON MISTAKES

MISTAKE: [X] SVM chooses any separating line. Wrong. It chooses the one with maximum margin.

MISTAKE: [X] All points determine the classifier. Wrong. Only Support Vectors determine the optimal hyperplane.

MISTAKE: [X] Hyperplane means only a line. Wrong. A line is only a 2D hyperplane. Our project uses a 17-dimensional hyperplane.

REVISION NOTES

Remember these five ideas.

Decision Boundary — Separates Classes.

Margin — Safety Distance.

Maximum Margin — Better Generalization.

Support Vectors — Nearest Points.

Hyperplane — Decision Boundary in Higher Dimensions.

MEMORY TRICK

Remember this sentence. "Support Vectors choose the Hyperplane that gives the Maximum Margin." This one sentence summarizes the entire philosophy of SVM.

KNOWLEDGE CHECK

Level 1

- What is SVM?
- What is a decision boundary?
- What is a margin?
- What are support vectors?
- What is a hyperplane?

Level 2

- Why does SVM maximize the margin?
- Why don't all training samples affect the hyperplane equally?
- Why is the decision boundary called a hyperplane in our project?
- How many dimensions does the hyperplane have in our implementation?
- Why are support vectors so important?

Professor Level

- Explain why maximizing the margin improves generalization.
- What would happen if one support vector were removed from the training data?
- Why is SVM particularly suitable for high-dimensional feature spaces like the 17-feature vectors in your project?
- Explain the complete meaning of the name Support Vector Machine.
- How does the concept of a hyperplane change as the number of features increases?
- End of Chapter 12 (Part 1)
- Congratulations.
- You now understand the core philosophy of SVM.
- Notice something important:
- Up to this point, we have not discussed a single equation.
- That's intentional.
- If you first understand the idea, the mathematics becomes easy.

Author's Note The next part is where SVM becomes even more interesting. We'll answer questions like: Why can't a straight line separate every dataset? What happens when the classes overlap? What are Hard Margin and Soft Margin? What is the C parameter? Why did our project use the RBF kernel instead of a linear kernel? How do kernels transform the data without explicitly increasing dimensions? This is the section that most students find difficult—but by connecting everything back to your 17-feature fault-classification project, it becomes much easier to understand.

CHAPTER 12 Part 2 — Hard/Soft Margin, Kernels, RBF

Chapter 12 — Support Vector Machine (SVM) Part 2 — Hard Margin, Soft Margin, Kernels and Why We Chose RBF Before Starting

In Part 1

we learned Decision Boundary Margin Support Vectors Hyperplane Now we ask one very important question. Question. Can SVM always separate classes using one straight line? Most beginners say Yes. Wrong. Let's see why.

First Question

Suppose our data looks like this.

Peak ↑



Question. Can one straight line separate them? Yes. Easy. Now look at another dataset.

Peak ↑



Question. Can one straight line separate these? No. No matter where you draw, some points will always be on the wrong side.

PROFESSOR ASKS

Can linear SVM classify every dataset?

EXCELLENT ANSWER

No. Linear SVM works only when the classes are approximately linearly separable. More complex datasets require nonlinear decision boundaries using kernel functions. Excellent. Remember approximately linearly separable. Hard Margin SVM Let's imagine an ideal world. No noise. No mistakes. Perfect data. Question. Can SVM separate every point perfectly? Yes. That is called Hard Margin SVM. Example ● ● ●



▲ ▲ ▲ Every point is classified correctly. No exceptions.

PROFESSOR ASKS

What is Hard Margin SVM?

EXCELLENT ANSWER

Hard Margin SVM assumes that the training data is perfectly linearly separable and does not allow any misclassification. Problem with Hard Margin Question. Is real-world data perfect? No. Question. Why? Real data contains Noise. Measurement errors. Outliers. Component tolerances. Question.

Can Hard Margin

handle them? No. It becomes too strict.

Real Life Analogy

Imagine a teacher who says Every answer must be 100% perfect. No spelling mistakes. No calculation mistakes. Question. Will many students pass? Probably not.

Hard Margin

is like that teacher. Soft Margin SVM Now let's become more realistic. Suppose one point is slightly on the wrong side. Should we reject the entire classifier? No. Instead we allow a few mistakes if the overall decision boundary becomes better. That is Soft Margin SVM. Example ● ● ●

▲ ● ▲ ▲ ▲ Notice one point is misclassified. That's acceptable if generalization improves.

PROFESSOR ASKS

What is Soft Margin SVM?

EXCELLENT ANSWER

Soft Margin SVM allows limited misclassification of training samples in order to achieve better generalization on unseen data. Excellent. Why Soft Margin is Better Question. Would you rather memorize every training sample or perform well on new data? Obviously new data.

Soft Margin

focuses on generalization. Exactly what our project needs.

The C Parameter

Now one of the most important SVM parameters. Question. How many mistakes should

Soft Margin

allow? Answer. That is controlled by C. Question. Suppose C is very large. What happens? SVM says Don't make mistakes. Try to classify everything correctly. Very strict. Question. Suppose C is very small. Now SVM says A few mistakes are okay. Focus on generalization.

PROFESSOR ASKS

What is the C parameter?

EXCELLENT ANSWER

The C parameter controls the trade-off between maximizing the margin and minimizing classification errors on the training data. This answer is extremely important.

Real Life Analogy

Imagine a school.

Strict Principal

Every mistake gets punished. High C.

Friendly Principal

Small mistakes are acceptable. Low C. Exactly the same idea.

In Our Project

Question. Did we manually choose C? No. Very important. We used GridSearchCV to find the best value. We'll study GridSearchCV later. Remember this. We did not guess the value. Now the Biggest Question. Suppose the classes look like this. ● ● ● ● ● ● ▲ ● ▲ ▲ ▲ Question. Can any straight line separate them? Impossible. Need something smarter.

The Kernel Trick

This is the most famous idea in SVM. Question. Instead of changing the data, what if we change the space in which the data exists? That is the Kernel Trick.

Simple Analogy

Imagine a flat sheet of paper. On it lie a blue circle surrounded by red circles. Question. Can you separate them using one straight line? No. Now lift the center of the paper. Suddenly the blue point moves upward. Now one plane can separate them. Exactly what kernels do.

PROFESSOR ASKS
What is the Kernel Trick?
EXCELLENT ANSWER

The Kernel Trick implicitly maps data into a higher-dimensional feature space where classes become easier to separate without explicitly computing the higher-dimensional coordinates. Excellent.

Important Point

Notice something. The kernel does NOT change our 17 features. It changes how SVM looks at those features. Very important. Types of Kernels Many kernels exist. The common ones are Linear Polynomial RBF Sigmoid Let's understand each one.

Linear Kernel

Simplest. Uses a straight line (or hyperplane). Good when data is already linearly separable. Question. Did we use Linear Kernel? No. Because our fault classes were not perfectly linear.

Polynomial Kernel

Instead of a straight boundary, it creates curved boundaries. Useful for moderately complex datasets. Question. Did we use it? No. GridSearchCV tested it, but it wasn't our final choice. (We'll verify this when we discuss your exact GridSearchCV results.) RBF Kernel Now the most important one. This is the kernel used in our project. Question. Why? Because our 17-dimensional fault data has complex nonlinear relationships. The RBF Kernel creates very flexible decision boundaries. It can separate complex patterns that a straight line cannot.

PROFESSOR ASKS
Why did you choose the RBF kernel?
EXCELLENT ANSWER

The fault classes in our feature space were not linearly separable. The RBF kernel can model complex nonlinear relationships, making it well suited for analog circuit fault classification. Excellent.

Real Life Analogy

Imagine drawing a fence around a tree. Question. Will a straight fence fit perfectly? No. Need a curved fence. RBF creates that flexible boundary. Gamma Now another parameter. Question. How flexible should the RBF boundary be? Answer. That is controlled by Gamma.

Small Gamma

Smooth boundary.

Large Gamma

Highly complex boundary.

PROFESSOR ASKS

What does Gamma control?

EXCELLENT ANSWER

Gamma controls the influence of individual training samples in the RBF kernel. Higher values produce more complex decision boundaries, while lower values produce smoother boundaries.

In Our Project

Again did we manually choose Gamma? No. GridSearchCV selected the optimal value during hyperparameter tuning.

Connection to Our Project

Our pipeline now becomes

17 Features

StandardScaler — SVM. RBF Kernel

Optimal Hyperplane

Fault Prediction

Notice The kernel works inside the SVM. We never explicitly transform the features ourselves.

COMMON MISTAKES

MISTAKE: [X] Kernel creates new features. Wrong. It changes the mathematical space, not the original feature vector.

MISTAKE: [X] Large C always means better accuracy. Wrong. Too large can cause overfitting.

MISTAKE: [X] Large Gamma is always better. Wrong. Too large can also overfit.

MISTAKE: [X] Hard Margin works well for noisy data. Wrong. Soft Margin is designed for noisy, real-world datasets.

REVISION NOTES

Remember these ideas.

Hard Margin — No mistakes allowed.

Soft Margin — Some mistakes allowed.

C — Strictness.

Kernel — Changes the feature space.

RBF — Best for nonlinear data.

Gamma — Controls boundary complexity.

MEMORY TRICK

Remember this sentence. "C controls mistakes. Gamma controls flexibility. RBF handles nonlinear patterns." One sentence. Three important concepts.

KNOWLEDGE CHECK

Level 1

- What is Hard Margin SVM?
- What is Soft Margin SVM?
- What does the C parameter control?
- What is a kernel?
- Why did we choose the RBF kernel?

Level 2

- Why isn't Linear SVM suitable for every dataset?
- Why is Soft Margin better for real-world applications?
- How does Gamma affect the decision boundary?
- Why didn't we manually choose C and Gamma?
- How does the Kernel Trick help SVM classify nonlinear data?

Professor Level

- Explain the trade-off controlled by the C parameter.
- Why is the RBF kernel suitable for analog fault diagnosis?
- What happens if Gamma is set extremely high?
- Why is the Kernel Trick computationally efficient?
- How would classifier behavior change if a Linear kernel were used instead of RBF on your dataset?
- End of Chapter 12 (Part 2)
- Congratulations.
- You now understand why SVM is powerful, why nonlinear kernels are needed, and why your project used the RBF kernel with tuned hyperparameters instead of a simple linear classifier.

Author's Note The next part will move from SVM theory to your actual implementation. We'll explain: How your Python code creates the SVM model. Every parameter in your SVC(.) call. Why probability=True (if used) matters. How multiclass classification works using One-vs-One, which is especially relevant to your fault classes. How the model actually predicts one of your circuit fault labels from the 17-feature vector. This is where the theory and your code finally come together.

CHAPTER 12 Part 3 — Our SVM Implementation

Chapter 12 — Support Vector Machine (SVM) Part 3 — Our SVM Implementation (Line-by-Line Explanation) Before Starting In the previous parts, we studied the complete theory behind SVM. Now we leave the textbook and enter our actual project. This chapter explains what our Python code is doing line by line. This is one of the most important chapters for Viva Placements Interviews Project Demonstration because now you can explain your own code, not just SVM theory. Where Are We in the Pipeline? Let's remember our complete project. **LTspice** — CSV Files.

Feature Extraction

17 Features

Feature Matrix (X)

Labels (y)

Train-Test Split

StandardScaler — SVM ← We are here. **Prediction** — Evaluation. Everything before SVM is already complete. Now we train the classifier.

Step 1 — Importing SVM

Our code first imports the SVM classifier. Example

```
from sklearn.svm import SVC
```

Question. What is SVC? SVC stands for Support Vector Classifier. This is Scikit-learn's implementation of SVM for classification problems.

PROFESSOR ASKS

What is SVC?

EXCELLENT ANSWER

SVC is Scikit-learn's implementation of the Support Vector Machine algorithm for classification tasks.

Step 2 — Creating the Model

Now our code creates the classifier. Conceptually it looks like

```
model = SVC(...)
```

Question. Has

Machine Learning

started learning? No. Very important. At this stage, we have only created an empty model. Think of it like buying an empty notebook. Question. Has the notebook learned anything? Obviously not.

Real Life Analogy

Imagine you buy a new laptop. Question. Is it already trained to classify faults? No. Creating the SVC object only prepares the algorithm. Learning begins later.

Step 3 — Choosing Parameters

Inside SVC(.) we specify important parameters. Example kernel = "rbf" Question. Why RBF? Because our dataset contains nonlinear relationships between the 17 features and the fault classes. We already studied this in Part 2.

PROFESSOR ASKS

Why RBF?

EXCELLENT ANSWER

The RBF kernel can model nonlinear decision boundaries, making it suitable for our analog fault classification problem.

Step 4 — Hyperparameters

Question. Did we randomly choose the values of C Gamma No. Very important. Our project used GridSearchCV to determine the best combination. So the values used inside SVC(.) were already optimized. We will study GridSearchCV in detail later.

Step 5 — Training the Model

Now the most important line. Conceptually our code executes `model.fit(X_train, y_train)` This line changes everything. Question. What happens here? This is where

Machine Learning

actually begins.

PROFESSOR ASKS

What does fit() do?

EXCELLENT ANSWER

The fit() function trains the SVM model using the training feature vectors and their corresponding labels. Very important. Remember fit means training. What Happens Inside fit()? Students usually think fit() is magic. It isn't. Internally, SVM starts looking at thousands of training samples. Each sample contains

17 Features

Fault Label

Example Peak 5.2 RMS 3.8. DC Level **4.9** — RC_Normal. Then another sample. Then another. Eventually SVM finds the optimal hyperplane. What Does fit() Produce? Question. After training, what do we get? Do we get accuracy? No. Do we get predictions? No. We get a trained model. Think of a student. Question. After studying for six months, what has changed? Knowledge. Not exam marks. The exam comes later. Exactly the same.

Step 6 — Making Predictions

Now comes the exam. Our code uses `predictions = model.predict(X_test)` Question. What is X_test? It contains feature vectors the model has never seen. Exactly what we want.

PROFESSOR ASKS

What does predict() do?

EXCELLENT ANSWER

The predict() function uses the trained SVM model to classify previously unseen feature vectors. Inside predict() Suppose the test sample looks like Peak 5.1 RMS 3.7. DC Level 0.001 Question. What does SVM do? It compares this point with the learned decision boundaries and predicts one class. Example **Prediction** — RC_Open. Simple. Difference Between fit() and predict() Students often confuse these. Remember this table.

fit()	Learn from training data
predict()	Classify new data

One learns. The other uses what was learned. Why Don't We Predict on Training Data? Question. Suppose we predict using `X_train`. Question. Will accuracy be high? Usually Yes. Question. Does that prove the model is good? No. Because the model has already seen that data. Real performance must be measured using unseen data.

Real Life Analogy

Suppose your teacher gives you the answer sheet before the exam. Question. Can you score 100%? Probably yes. Question. Does that prove you understand the subject? No. Testing must use new questions. Exactly why we use `X_test`.

In Our Project

This sequence is exactly what happened. **`X_train`** — `fit()`. **Trained SVM** — `predict()`.

Predicted Fault Class

Notice

Feature Extraction

was already complete. SVM never saw the waveform. It only saw the 17-dimensional feature vectors. What Does the Model Store? Many students ask this. Question. After training, what is inside the model? It stores information such as Support Vectors Decision Boundaries Hyperplane Parameters Learned Relationships It does not store the waveform images or LTspice files.

PROFESSOR ASKS
What is stored after training?
EXCELLENT ANSWER

The trained SVM stores the learned decision boundaries, support vectors, and parameters required to classify new feature vectors.

Connection to Our Project

Everything we studied until now comes together. **LTspice** — CSV Files.

17 Features

StandardScaler — `SVM.fit()`.

Trained Model

`predict()`

Fault Prediction

This is the complete learning pipeline used in your project.

COMMON MISTAKES

MISTAKE: [X] Creating the SVM object means training. Wrong. Training begins only after `fit()`.

MISTAKE: [X] `predict()` trains the model. Wrong. `predict()` only uses the trained model.

MISTAKE: [X] Training and testing use the same data. Wrong. They must remain separate.

REVISION NOTES

Remember
these four steps.
Import — SVC.
Create — Empty Model.
fit() — Learning.
predict() — Prediction.

MEMORY TRICK

Remember this sentence. "Create → Train → Predict." That's the entire life cycle of an SVM model.

KNOWLEDGE CHECK

Level 1

- What is SVC?
- What does fit() do?
- What does predict() do?
- What is X_train?
- What is X_test?

Level 2

- Why is the model empty before fit()?
- Why is testing performed on unseen data?
- What information does the trained SVM store?
- Why is fit() called only once but predict() can be called many times?
- What is the difference between creating a model and training a model?

Professor Level

- Explain the complete SVM training pipeline used in your project.
- What happens internally when fit() is called?
- Why is feature extraction performed before SVM training?
- Why does SVM never directly process LTspice waveforms?
- What is the role of the training and testing datasets in ensuring model reliability?
- End of Chapter 12 (Part 3)
- Congratulations.
- You now understand not only the theory of SVM, but also how your implementation actually trains and uses the classifier. You can now explain the core SVC → fit() → predict() workflow confidently in a viva or interview.

Author's Note The next section is one of the strongest parts of your project:

Chapter 13 — GridSearchCV (Hyperparameter Tuning) This chapter will explain: Why we didn't manually choose C and Gamma. What hyperparameters are. How GridSearchCV systematically tests combinations. Cross-validation. Why GridSearchCV is better than trial-and-error. How it selected the best SVM configuration for your analog fault diagnosis project. The exact flow followed by your implementation. This chapter is extremely valuable because it demonstrates that your model was systematically optimized, rather than relying on arbitrary parameter choices.